Experiment 13: Sequential Dataset Preparation and RNN Prediction

Question

Problem Statement: A state electricity board must forecast next-hour power demand to schedule generation and avoid shortages. Hourly electricity consumption data for one year (or a daily temperature series as a substitute). The objective is to convert a time series into supervised sequences and predict the next value with a SimpleRNN. Use TensorFlow, Keras, NumPy, Pandas, Matplotlib.

Tasks:

1. Load the series with Pandas; handle missing values and scale it.
2. Create sliding windows (for example, 24 past hours → next hour).
3. Split chronologically into train and test sets.
4. Build and train a SimpleRNN model in Keras.
5. Plot actual against predicted demand and report MAE and RMSE.

Code

```
"""Experiment 13: Sequential Dataset Preparation and RNN Prediction (hourly electricity demand)"""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

np.random.seed(42); tf.random.set_seed(42)

# ---------------- Task 1: load series, handle missing values, scale ----------------
# Synthetic hourly demand (MW) for one year: daily cycle + weekly cycle + seasonal trend + noise
hours = pd.date_range("2023-01-01", periods=8760, freq="h")
t = np.arange(8760)
demand = (500 + 150 * np.sin(2 * np.pi * (t % 24 - 6) / 24)          # daily peak in the evening
          + 40 * (hours.dayofweek < 5)                               # weekdays use more
          + 80 * np.sin(2 * np.pi * t / 8760)                        # seasonal trend
          + np.random.normal(0, 15, 8760))
df = pd.DataFrame({"demand": demand}, index=hours)
df.loc[df.sample(frac=0.01, random_state=1).index, "demand"] = np.nan   # 1% missing values
# To use your own file:  df = pd.read_csv("demand.csv", parse_dates=["time"], index_col="time")
print("Missing values before:", df["demand"].isna().sum())
df["demand"] = df["demand"].interpolate(method="time")                  # fill gaps
print("Missing values after :", df["demand"].isna().sum())
print(df.head())

split = int(0.8 * len(df))                                              # chronological split point
scaler = MinMaxScaler().fit(df.iloc[:split])                            # fit on TRAIN only (no leakage)
scaled = scaler.transform(df).ravel()

# ---------------- Task 2: sliding windows (24 past hours -> next hour) ----------------
WINDOW = 24
X, y = [], []
for i in range(len(scaled) - WINDOW):
    X.append(scaled[i:i + WINDOW]); y.append(scaled[i + WINDOW])
X = np.array(X)[..., np.newaxis]            # (samples, 24, 1)
y = np.array(y)
print("X shape:", X.shape, "y shape:", y.shape)

# ---------------- Task 3: chronological train/test split (no shuffling) ----------------
cut = split - WINDOW
X_train, y_train, X_test, y_test = X[:cut], y[:cut], X[cut:], y[cut:]
print("Train:", X_train.shape, "Test:", X_test.shape)

# ---------------- Task 4: SimpleRNN ----------------
model = keras.Sequential([layers.Input((WINDOW, 1)), layers.SimpleRNN(32), layers.Dense(1)])
model.compile(optimizer=keras.optimizers.Adam(0.001), loss="mse")
```

```
model.summary()
hist = model.fit(X_train, y_train, validation_split=0.1, epochs=15, batch_size=64, verbose=2)


plt.figure(figsize=(7, 4))
plt.plot(hist.history["loss"], label="train"); plt.plot(hist.history["val_loss"], label="validation")
plt.xlabel("Epoch"); plt.ylabel("MSE (scaled)"); plt.title("SimpleRNN training curve"); plt.legend();
plt.show()


# ---------------- Task 5: actual vs predicted, MAE & RMSE ----------------
pred = model.predict(X_test, verbose=0)
pred_mw = scaler.inverse_transform(pred).ravel()                          # back to MW
actual_mw = scaler.inverse_transform(y_test.reshape(-1, 1)).ravel()
mae = mean_absolute_error(actual_mw, pred_mw)
rmse = np.sqrt(mean_squared_error(actual_mw, pred_mw))
print(f"Test MAE  = {mae:.2f} MW")
print(f"Test RMSE = {rmse:.2f} MW")


plt.figure(figsize=(12, 4.5))
plt.plot(actual_mw[:168], label="Actual"); plt.plot(pred_mw[:168], "--", label="Predicted")
plt.xlabel("Hour (first week of test set)"); plt.ylabel("Demand (MW)")
plt.title(f"Actual vs predicted demand (MAE={mae:.1f}, RMSE={rmse:.1f})"); plt.legend(); plt.show()
```

Output

```
print(f"Test RMSE = {rmse:.2f} MW")

plt.figure(figsize=(12, 4.5))
plt.plot(actual_mw[:168], label="Actual"); plt.plot(pred_mw[:168], "--", label="Predicted")
plt.xlabel("Hour (first week of test set)"); plt.ylabel("Demand (MW)")
plt.title(f"Actual vs predicted demand (MAE={mae:.1f}, RMSE={rmse:.1f})"); plt.legend(); plt.show()
```

```
Missing values before: 88
Missing values after : 0
                     demand
2023-01-01 00:00:00  357.450712
2023-01-01 01:00:00  353.094542
2023-01-01 02:00:00  379.926279
2023-01-01 03:00:00  416.951573
2023-01-01 04:00:00  421.717222
X shape: (8736, 24, 1) y shape: (8736,)
Train: (6984, 24, 1) Test: (1752, 24, 1)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 32)	1,088
dense (Dense)	(None, 1)	33

```
Total params: 1,121 (4.38 KB)
Trainable params: 1,121 (4.38 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/15
99/99 - 3s - 30ms/step - loss: 0.1322 - val_loss: 0.0281
Epoch 2/15
99/99 - 1s - 6ms/step - loss: 0.0070 - val_loss: 0.0042
Epoch 3/15
99/99 - 1s - 6ms/step - loss: 0.0025 - val_loss: 0.0032
Epoch 4/15
99/99 - 1s - 6ms/step - loss: 0.0019 - val_loss: 0.0026
Epoch 5/15
99/99 - 1s - 6ms/step - loss: 0.0016 - val_loss: 0.0022
Epoch 6/15
99/99 - 1s - 6ms/step - loss: 0.0015 - val_loss: 0.0018
Epoch 7/15
99/99 - 1s - 6ms/step - loss: 0.0014 - val_loss: 0.0016
Epoch 8/15
99/99 - 1s - 6ms/step - loss: 0.0013 - val_loss: 0.0015
Epoch 9/15
```

